SNOWFLAKE DATA ENGINEERING PROJECT

# Healthcare Intelligence Hub

Patient, Cost & Care Analytics

## Phase 2 — Requirement Analysis

Institutional template-based phase document

| | |
|---|---|
| Student | Aashi Bindal |
| Enrollment No. | 0827CI241004 |
| Team ID | [Team ID] |
| Institution | Acropolis Institute of Technology and Research · Department of CSIT |
| Faculty Guide | [Faculty Guide] |
| Academic Session | [Academic Session] |
| Document date | October 2026 |

Prepared from the verified project source (query.sql, Health/streamlit_app.py, Health/snowflake.yml, six CSV datasets). Bracketed fields are placeholders for information not supplied.

# Contents

**2.1 Problem Statement and Business Objectives** ..... 1

**2.2 Customer Journey Map** ..... 2

**2.3 Solution Requirements (Functional & Non-functional)** ..... 2

2.3.1 Stakeholder requirements ..... 4

2.3.2 User roles and use cases ..... 4

2.3.3 Source data and data requirements ..... 4

2.3.4 Business questions the solution answers ..... 5

2.3.5 Data quality requirements ..... 5

2.3.6 Hardware and software requirements ..... 5

2.3.7 Constraints, risks, assumptions and acceptance criteria ..... 6

**2.4 Data Flow Diagram & User Stories** ..... 6

**2.5 Technology Stack (Architecture & Stack)** ..... 8

# Phase 2 — Requirement Analysis

**Project:** Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Templates covered:** Customer Journey Map · Solution Requirements (4 marks) · Data Flow Diagram & User Stories (4 marks) · Technology Stack (2 marks)

> **Template note.** The Drive folder lists `Customer Journey Map.pdf`, but that file was not among the uploads. Section 2.2 therefore follows the conventional journey-map layout (stages × actions, touchpoints, emotions, pain points, opportunities). Replace it with the official layout if it differs. The other three templates were received; their headings and table columns are reproduced exactly. They carry the printed label "Project Design Phase-II".

## 2.1 Problem Statement and Business Objectives

**Problem statement (official):** Hospital administrators and finance teams need a consolidated, queryable view of patient visits — spanning cost, length of stay, satisfaction and readmissions — that today sits fragmented across raw operational records. Without a structured warehouse, it is difficult to answer basic questions such as which facilities and doctors are performing well, which diagnoses drive the longest and most expensive stays, and where patient satisfaction is at risk.

The statement poses three questions. Each maps to project components as follows:

*Table 2.0 — Problem-statement questions and how they are answered*

| # | Question in the problem statement | How the solution answers it | Finding (full dataset, local replica) |
|---|---|---|---|
| Q1 | Which facilities and doctors are performing well? | `VW_FACILITY_ANALYTICS`, `VW_DOCTOR_ANALYTICS`; app facility chart and table, doctor leaderboard (visits, avg cost, satisfaction, readmissions) | Facility volume 958–1,039 visits; City General Hospital busiest (1,039). Highest doctor satisfaction: Dr. Neha Brown, D002 (4.02) |
| Q2 | Which diagnoses drive the longest and most expensive stays? | `VW_DIAGNOSIS_ANALYTICS` (avg cost, avg LOS); app filter Visit Type = Inpatient with diagnosis charts | Inpatient stays: longest Fracture (8.60 days avg), Anemia (8.32), Diabetes (8.07); most expensive Kidney Disease (14,816.82 avg), Hypertension (14,640.75), Asthma (14,549.67) |
| Q3 | Where is patient satisfaction at risk? | Avg Satisfaction KPI; satisfaction columns in facility and doctor tables and in `VW_DIAGNOSIS_ANALYTICS` | Lowest facility: Sunrise Hospital (3.78; 13.9% of visits score 1–2). Lowest diagnosis: Asthma (3.75). Lowest doctors: Dr. Rohan Singh, D060 (3.65), Dr. Rohan Gupta, D004 (3.69) |

The project's business objectives are:

| ID | Objective | Implemented by |
|---|---|---|
| BO-1 | Consolidate patient, doctor, facility, diagnosis and visit data | `CURATED.FACT_PATIENT_VISITS` (LEFT JOIN of staging tables) |
| BO-2 | Analyse visit volumes, treatment costs and LOS | `VW_EXECUTIVE_KPI`, `VW_MONTHLY_VISITS`, `VW_VISIT_TYPE_DISTRIBUTION`; app KPI cards and trend |

| ID | Objective | Implemented by |
|---|---|---|
| BO-3 | Compare facilities, doctors and diagnoses | `VW_FACILITY_ANALYTICS`, `VW_DOCTOR_ANALYTICS`, `VW_DIAGNOSIS_ANALYTICS`; app facility chart and table, doctor leaderboard, top-10 diagnoses |
| BO-4 | Monitor satisfaction and 30-day readmission | `VW_READMISSION_ANALYTICS`; app readmission-rate card and chart, satisfaction card |
| BO-5 | Analyse insurance coverage, patient payment and cost breakdown | `VW_EXECUTIVE_KPI` totals; app cost-breakdown chart. **Partial:** no per-insurer view or chart exists |
| BO-6 | Present insights through an interactive, filterable Streamlit app | `Health/streamlit_app.py` (5 filters) |

## 2.2 Customer Journey Map

*(Official template not received; conventional structure used.)* Persona: hospital administrator / finance manager preparing the monthly performance review.

*Table 2.1 — Customer journey map*

| Stage | 1. Need arises | 2. Gather data | 3. Prepare & analyse | 4. Present & decide | 5. Follow up |
|---|---|---|---|---|---|
| **Actions (today)** | Month-end review is scheduled | Requests CSV exports from registration, HR roster, facility master, coding and billing | Joins files in spreadsheets; fixes inconsistent labels; computes averages | Shares static charts in a meeting | Repeats the cycle next month |
| **Touchpoints** | Calendar, email | Source systems, IT team | Excel / Sheets | Slides | Email |
| **Emotions** | Neutral | Impatient | Stressed | Anxious about accuracy | Fatigued |
| **Pain points** | Short deadline | Waiting on several teams | Manual joins, label mismatches | Cannot answer follow-up "what if" questions live | No reuse of prior work |
| **Opportunity (with the Hub)** | Self-service access | CSVs staged once into `HEALTH_DB.RAW` | STAGING cleans; CURATED joins; MART computes KPIs | Streamlit filters answer questions live | Re-run `query.sql` (or a future Task) to refresh |
| **Emotion (with the Hub)** | Confident | Relieved | In control | Credible | Efficient |

## 2.3 Solution Requirements (Functional & Non-functional)

| Field | Value |
|---|---|
| Date | 31 January 2026 *(as printed on template)* |
| Team ID | [Team ID] |
| Project Name | Healthcare Intelligence Hub: Patient, Cost & Care Analytics |

| Field | Value |
|---|---|
| Maximum Marks | 4 Marks |

**Functional Requirements:** Following are the functional requirements of the proposed solution.

*Table 2.2 — Functional requirements*

| FR No. | Functional Requirement (Epic) | Sub Requirement (Story / Sub-Task) | Priority (High/Medium/Low) |
|---|---|---|---|
| 1 | Data Access & Integration | Access six business datasets: PATIENTS (8 cols, 3,000 rows), DOCTORS (5 cols, 60), FACILITIES (4 cols, 10), DIAGNOSES (3 cols, 15), DATE (8 cols, 974) and PATIENT_VISITS (19 cols, 10,000). Integrate them in Snowflake `HEALTH_DB` through file format `RAW.CSV_FORMAT` (header skip, NULL_IF, `YYYY-MM-DD` dates) and internal stage `RAW.HEALTHCARE_STAGE` into typed `*_RAW` tables. | High |
| 2 | Data Processing | Trim all text keys and attributes; standardise case of categorical fields with `INITCAP`; remove rows with NULL business keys; build five deduplicated dimensions ( `SELECT DISTINCT` ); build `FACT_PATIENT_VISITS` at visit grain by LEFT JOIN to patient, doctor, facility and diagnosis attributes; run NULL-key, duplicate-key, row-count and orphan checks. | High |
| 3 | Business Analysis | Compute total visits, unique patients, doctors, facilities; total and average cost; total insurance coverage and patient payment; average satisfaction; average LOS; readmission count and rate (% of visits flagged "Yes"); monthly visit and cost trend; aggregates by diagnosis, facility, doctor, visit type, treatment status and admission status; average cost component by visit type. | High |
| 4 | Business Insights & Analytical Outputs | Ten MART views ( `VW_EXECUTIVE_KPI` … `VW_READMISSION_ANALYTICS` ) and a Streamlit dashboard with 6 KPI cards, monthly trend line, visit-type and admission-status donuts, top-10 diagnoses bar, facility bar plus table, treatment-status bar, readmission-rate-by-diagnosis bar, cost-breakdown stacked bar and a doctor leaderboard table. | High |
| 5 | User Interaction & Analysis | Sidebar filters for visit date range, facility, doctor specialty, visit type and diagnosis category, with an "All" option on each. Every KPI, chart and table recomputes on the filtered set. Chart tooltips show the underlying values. Tables are sortable (Streamlit `st.dataframe` ). | High |
| 6 | Data Update & Reporting | Re-running `query.sql` rebuilds STAGING and CURATED ( `CREATE OR REPLACE` ); MART views always read current CURATED data; the app caches the fact table for 600 s ( `@st.cache_data(ttl=600)` ), then re-reads it. *Proposed:* scripted `COPY INTO` and scheduled Tasks. | Medium |

**Non-functional Requirements:** Following are the non-functional requirements of the proposed solution.

*Table 2.3 — Non-functional requirements*

| FR No. | Non-Functional Requirement | Description |
|---|---|---|
| NFR-1 | Usability | Single-page wide layout; filters grouped in the sidebar; every chart titled with axis labels; KPI cards formatted with thousands separators and units ("/ 5", "%", "days"); a footer shows the filtered visit count and source table. |
| NFR-2 | Security | App uses the caller's Snowflake session ( `get_active_session()` ), with no credentials in code or config. Data stays inside Snowflake. *Proposed:* least-privilege roles and masking policies for patient names and date of birth (see Phase 3 §3.9). |

| FR No. | Non-Functional Requirement | Description |
|---|---|---|
| NFR-3 | Reliability | Deterministic, idempotent rebuild ( `CREATE … IF NOT EXISTS` / `CREATE OR REPLACE` ); built-in validation queries for NULLs, duplicates, row counts and orphans; app guards divide-by-zero when a filter returns no rows. |
| NFR-4 | Performance | MART queries and app filters should return in under 5 s on an X-Small warehouse for the current 10,000-row volume. The fact table is cached in the app for 10 minutes. *Measured locally only (Phase 6); Snowflake timings Not Executed.* |
| NFR-5 | Availability | Availability is inherited from the Snowflake service and warehouse auto-resume; the app is reachable to authorised Snowsight users whenever the warehouse can start. |
| NFR-6 | Scalability | Compute scales by resizing the warehouse independently of storage. *Limitation:* the app loads the whole fact table into pandas, so very large volumes should push aggregation to SQL/MART views (Phase 8). |

### 2.3.1 Stakeholder requirements

| Stakeholder | Requirement |
|---|---|
| Administrator | Facility and doctor comparison on visits, cost, LOS and satisfaction; date filtering |
| Finance | Total cost, insurance versus patient share, cost by component and visit type |
| Quality lead | Readmission rate by diagnosis and doctor; treatment status mix |
| Data engineer | Re-runnable scripts; layer separation; validation output |

### 2.3.2 User roles and use cases

| UC | Actor | Use case | Main flow |
|---|---|---|---|
| UC-1 | Data engineer | Load and transform data | Upload CSVs to stage → load RAW → run STAGING, CURATED and MART sections of `query.sql` → review validation results |
| UC-2 | Administrator | Review overall performance | Open app → read KPI cards → inspect monthly trend |
| UC-3 | Administrator | Compare facilities | Select date range → view facility bar and table → sort by average satisfaction |
| UC-4 | Finance analyst | Analyse cost drivers | Select visit type → read cost-breakdown chart → query `VW_EXECUTIVE_KPI` for insurance and patient totals |
| UC-5 | Quality lead | Investigate readmissions | Filter diagnosis category → view readmission-rate chart and doctor leaderboard readmissions |

### 2.3.3 Source data and data requirements

| Dataset | Grain | Key | Rows | Required fields |
|---|---|---|---|---|
| patients.csv | patient | patient_id | 3,000 | gender, date_of_birth, city, insurance_provider, registration_date |
| doctors.csv | doctor | doctor_id | 60 | doctor_name, specialty, experience_years, department |
| facilities.csv | facility | facility_id | 10 | facility_name, city, facility_type |
| diagnoses.csv | diagnosis | diagnosis_id | 15 | diagnosis_name, diagnosis_category |

| Dataset | Grain | Key | Rows | Required fields |
|---|---|---|---|---|
| date.csv | calendar day | date | 974 (2024-01-01 → 2026-08-31) | year, quarter, month, month_name, week, day_of_week, is_weekend |
| patient_visits.csv | visit | visit_id | 10,000 | 4 FKs, visit_date, visit_type, statuses, LOS, 5 cost fields, insurance_coverage, patient_payment, satisfaction, readmission flag |

The complete column list is in `DATA_DICTIONARY.md`.

### 2.3.4 Business questions the solution answers

1. How many visits, unique patients, doctors and facilities are there in a period?
2. What is the monthly trend in visits, cost and satisfaction?
3. Which visit types are most frequent and most expensive?
4. Which diagnoses have the highest volume, cost and LOS?
5. Which facilities handle the most visits, and how do they compare on average cost, satisfaction and LOS?
6. Which doctors have the most visits, and what are their average cost, satisfaction and readmissions?
7. What is the 30-day readmission rate overall and by diagnosis?
8. How is total cost split between consultation, medication, lab and room, and between insurance and the patient?
9. What is the mix of treatment outcomes and admission status?

### 2.3.5 Data quality requirements

| DQ-ID | Dimension | Rule |
|---|---|---|
| DQ-1 | Completeness | Business keys are never NULL in STAGING |
| DQ-2 | Uniqueness | Each business key is unique per table |
| DQ-3 | Referential integrity | Every visit FK matches a dimension row; every visit date exists in DATE |
| DQ-4 | Validity | Satisfaction 1–5; LOS ≥ 0; costs ≥ 0; readmission flag ∈ |
| DQ-5 | Consistency | TOTAL_COST = sum of the four components; INSURANCE_COVERAGE + PATIENT_PAYMENT = TOTAL_COST |
| DQ-6 | Conformity | Categorical labels consistently cased ( `INITCAP` ), with acronyms preserved |

### 2.3.6 Hardware and software requirements

| Type | Requirement |
|---|---|
| Client hardware | Any laptop or desktop with a modern browser and 4 GB+ RAM |
| Cloud platform | Snowflake account (trial is sufficient) with a role able to create databases and Streamlit apps; warehouse `COMPUTE_WH` (X-Small sufficient) |
| Streamlit runtime | Streamlit in Snowflake; Python `~=3.11.0` ; dependency `streamlit[snowflake]` (from `pyproject.toml` ); `snowflake.yml` names compute pool `SYSTEM_COMPUTE_POOL_CPU` |

| Type | Requirement |
|---|---|
| Optional local tools | Snowflake CLI (to deploy with `snow streamlit deploy`); Python 3.11 with pandas to run `tests/local_validation.py` |

### 2.3.7 Constraints, risks, assumptions and acceptance criteria

| Type | Item |
|---|---|
| Constraint | Data limited to supplied CSVs; no live hospital-system feed |
| Constraint | Streamlit in Snowflake only allows packages available to its runtime |
| Risk | Manual loading step may be skipped or repeated (duplicates). Mitigation: `CREATE OR REPLACE` RAW tables before each load; row-count checks |
| Risk | Patient names in fact table expose PII. Mitigation (proposed): masking policy, analyst role on MART only |
| Assumption | Readmission flag is calculated correctly upstream |
| Acceptance | All DQ-1 to DQ-5 checks return 0 failures; KPI view matches an independent recomputation; all 5 filters change all visuals; app loads without error in Snowsight |

## 2.4 Data Flow Diagram & User Stories

| Field | Value |
|---|---|
| Date | 31 January 2026 *(as printed on template)* |
| Team ID | [Team ID] |
| Project Name | Healthcare Intelligence Hub: Patient, Cost & Care Analytics |
| Maximum Marks | 4 Marks |

**Data Flow Diagrams:** A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. It shows how data enters and leaves the system, what changes the information, and where data is stored.

**DFD Symbol Legend**

| Symbol | Name | Description |
|---|---|---|
| Oval / Rounded shape | External Entity | A person, organisation, or system outside the project that sends or receives data (here: CSV source extracts; administrator / finance user). |
| Rectangle with numbered header | Process | An activity that transforms incoming data into outgoing data (1.0 Stage & load … 5.0 Filter & visualise). |
| Rectangle (solid fill, no number) | Data Store | A place where data is held for later use (D1 RAW … D4 MART). |
| Labeled arrow | Data Flow | The movement of data between entities, processes and data stores. |

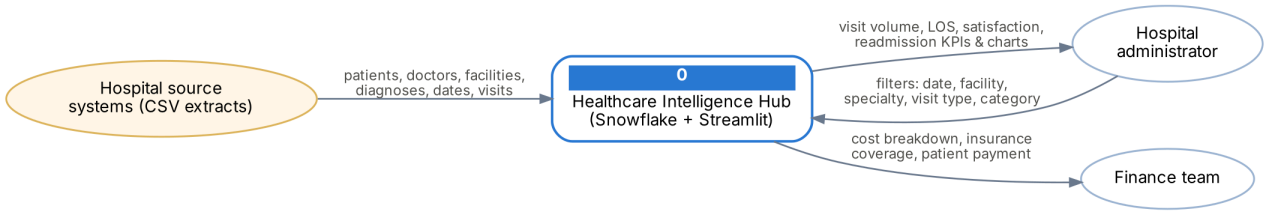



*Figure 2.1 — Context diagram (DFD Level 0)*

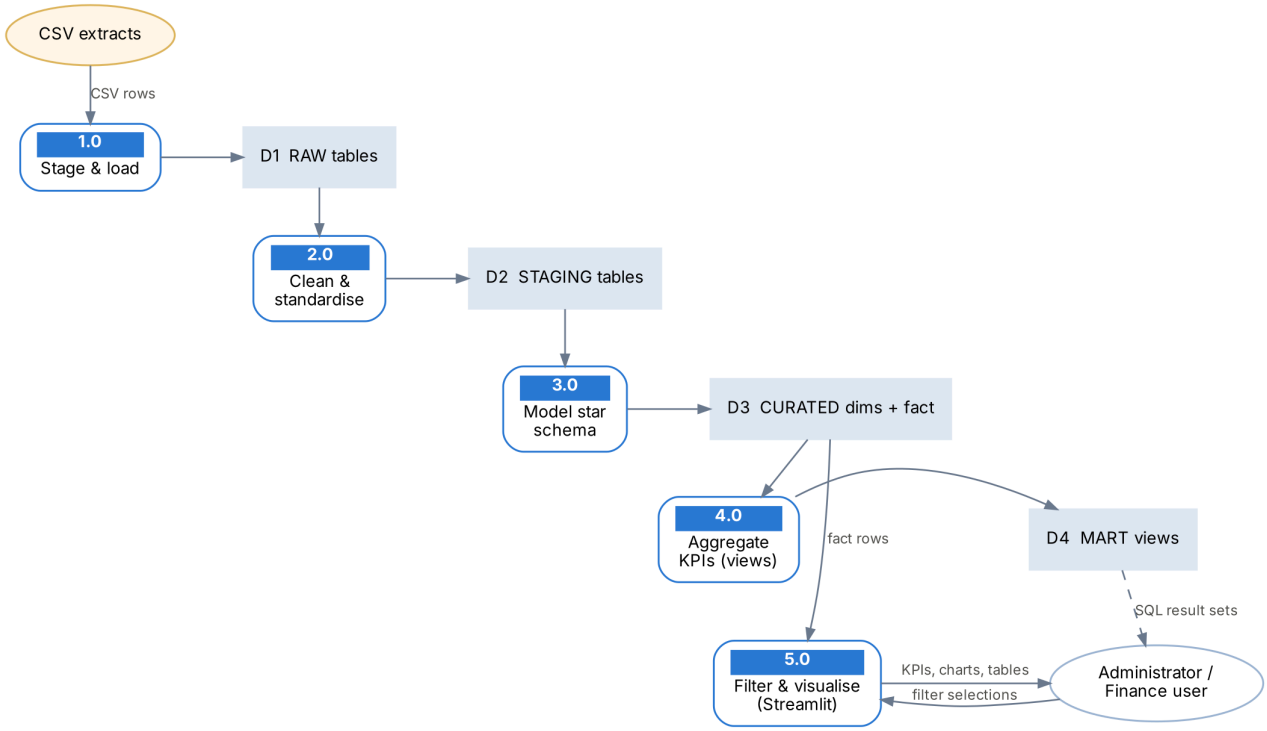



*Figure 2.2 — DFD Level 1: processes 1.0–5.0 and data stores D1–D4, as implemented in query.sql and streamlit_app.py*

**User Stories:** Use the below template to list all the user stories for the product.

*Table 2.4 — User stories*

| User Type | Functional Requirement (Epic) | User Story Number | User Story / Task | Acceptance criteria | Priority | Release |
|---|---|---|---|---|---|---|
| Data Engineer | Data Access & Integration | USN-1 | As a data engineer, I can create `HEALTH_DB` with RAW, STAGING, CURATED and MART schemas. | `SHOW SCHEMAS IN DATABASE HEALTH_DB` lists all four | High | Sprint-1 |
| Data Engineer | Data Access & Integration | USN-2 | As a data engineer, I can define a CSV file format and an internal stage, and load the six CSVs into typed RAW tables. | RAW row counts = 3,000 / 60 / 10 / 15 / 974 / 10,000 | High | Sprint-1 |
| Data Engineer | Data Processing | USN-3 | As a data engineer, I can build cleaned STAGING tables with trimmed, case-standardised values and non-NULL keys. | Null-key and duplicate-key checks return 0 | High | Sprint-2 |

| User Type | Functional Requirement (Epic) | User Story Number | User Story / Task | Acceptance criteria | Priority | Release |
|---|---|---|---|---|---|---|
| Data Engineer | Data Processing | USN-4 | As a data engineer, I can build five dimensions and a visit-grain fact table. | FACT rows = 10,000; orphan checks return 0 | High | Sprint-2 |
| Hospital Administrator | Business Analysis | USN-5 | As an administrator, I can see total visits, unique patients, total cost, average satisfaction, readmission rate and average LOS. | Six KPI cards render, matching `VW_EXECUTIVE_KPI` | High | Sprint-3 |
| Hospital Administrator | Business Insights | USN-6 | As an administrator, I can view the monthly visit trend and visit-type split. | Line chart and donut render | High | Sprint-3 |
| Hospital Administrator | Business Insights | USN-7 | As an administrator, I can compare facilities on visits, average cost and satisfaction. | Facility bar and table render, sorted by visits | High | Sprint-3 |
| Hospital Administrator | Business Insights | USN-8 | As an administrator, I can see a doctor leaderboard with visits, average cost, satisfaction and readmissions. | Table lists every doctor | Medium | Sprint-3 |
| Finance Analyst | Business Analysis | USN-9 | As a finance analyst, I can see average consultation, medication, lab and room cost by visit type. | Stacked bar renders | High | Sprint-3 |
| Finance Analyst | Business Analysis | USN-10 | As a finance analyst, I can query total insurance coverage and patient payment. | `VW_EXECUTIVE_KPI` returns both totals | Medium | Sprint-3 |
| Quality Lead | Business Insights | USN-11 | As a quality lead, I can see readmission rate by diagnosis. | Top-10 bar of readmission % renders | High | Sprint-3 |
| All dashboard users | User Interaction | USN-12 | As a user, I can filter by date range, facility, specialty, visit type and diagnosis category. | All KPIs and charts change with filters | High | Sprint-4 |
| Data Engineer | Data Update & Reporting | USN-13 | As a data engineer, I can deploy the app with `snowflake.yml`. | App opens in Snowsight | Medium | Sprint-4 |
| Administrator (security) | Data Update & Reporting | USN-14 | As an administrator, I can restrict access by role and mask patient names. | *Proposed: not implemented* | Low | Future |

## 2.5 Technology Stack (Architecture & Stack)

| Field | Value |
|---|---|
| Date | 31 January 2026 *(as printed on template)* |

| Field | Value |
|---|---|
| Team ID | [Team ID] |
| Project Name | Healthcare Intelligence Hub: Patient, Cost & Care Analytics |
| Maximum Marks | 2 Marks |

**Define Your Technology Stack:** Identify the technologies, platforms, databases, BI tools, and integrations used to build the Business Intelligence solution.

*Table 2.5 — Technology stack*

| S.No | Component | Description | Technology | Justification / Purpose |
|---|---|---|---|---|
| 1. | Programming / Query Language | Language used to query, filter, transform, aggregate, and analyse data. | **SQL (Snowflake dialect)**; **Python 3.11** | SQL implements all layers and KPI views ( `query.sql` ); Python drives the Streamlit app and in-app aggregations (pandas). |
| 2. | Database / Data Warehouse | Warehouse used to store and manage structured business data for analysis and reporting. | **Snowflake**: database `HEALTH_DB` ; schemas `RAW` , `STAGING` , `CURATED` , `MART` ; warehouse `COMPUTE_WH` | Separates storage and compute, gives full SQL, and runs Streamlit natively. |
| 3. | Data Ingestion | Used to load data from source files or external systems into the analytical environment. | **Snowflake internal stage** `RAW.HEALTHCARE_STAGE` + **file format** `RAW.CSV_FORMAT` ; CSV source files | Standard, typed CSV loading. *Note:* the `PUT` / `COPY INTO` commands are not in `query.sql` , so the load was presumably done through the Snowsight UI (not verifiable from the files). |
| 4. | Data Transformation and Business Logic | Tools/logic used to clean, join, filter, aggregate data and calculate KPIs, metrics and business rules. | **Snowflake SQL**: CTAS ( `TRIM` , `INITCAP` , `WHERE … IS NOT NULL` , `SELECT DISTINCT` , `LEFT JOIN` ), views ( `COUNT` , `SUM` , `AVG` , `ROUND` , `CASE` , `DATE_TRUNC` ); **pandas** in the app | Logic is versionable SQL, close to the data; pandas recomputes metrics after filtering. |
| 5. | Data Visualization / Analytics | Used to create interactive dashboards, charts, reports, KPIs and visual analysis. | **Altair** (line, arc/donut, bar, stacked bar) + **Streamlit** `st.metric` , `st.dataframe` | Declarative charts with tooltips, bundled with Streamlit. |
| 6. | Front-End / Interface | User interface through which users interact with dashboards, filters, parameters and visual insights. | **Streamlit in Snowflake** ( `streamlit_app.py` , deployed by `snowflake.yml` , title "HEALTH") | Native to Snowflake, no separate hosting, uses the user's Snowflake session. Tableau was considered but not used. |